Overflow، و printf بالفاصلة العشرية، ورموز الهروب

الدرس 5 — 1IOT، SST-ICT-01 — حدود char، %f، \a \t \n

22/09/2026

1 مراجعة سريعة

مراجعة لأنواع المتغيرات الثلاثة التي رأيناها حتى الآن (char من **8** **بتات**، وint من 16 بت، وfloat بالفاصلة العشرية) ولعوامل **الحساب** و**الإسناد** و**المقارنة** التي قدمناها سابقًا.

2 مشكلة overflow: عندما يكون نوع البيانات صغيرًا جدًا

مثال

واجب منزلي: أنشئ **متغيرين** prezzo (السعر) وquantità (الكمية)، واحسب حاصل ضربهما في totale (المجموع)، واطبع النتيجة.

```
char prezzo, quantita;
prezzo = 13;
quantita = 22;

char totale;
totale = prezzo * quantita;

printf("%d", totale);
```

بما أن 13 و22 عددان صغيران، يبدو نوع **char** كافيًا لاحتوائهما.

تنبيه

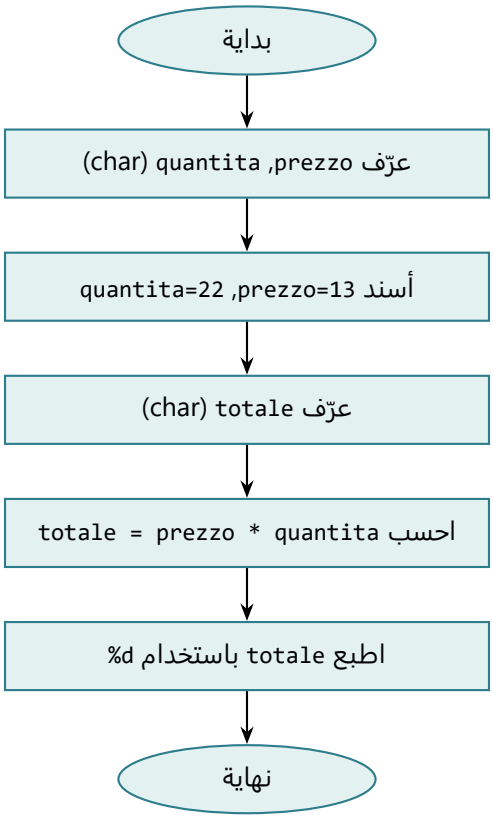
المشكلة: بدلًا من الحصول على 286 (أي 22×13) كنتيجة، كان البرنامج يطبع **30**. ولم تكن الآلة تشير إلى أي **خطأ**: كان البرنامج "يعمل"، لكن النتيجة كانت خاطئة.

تعريف

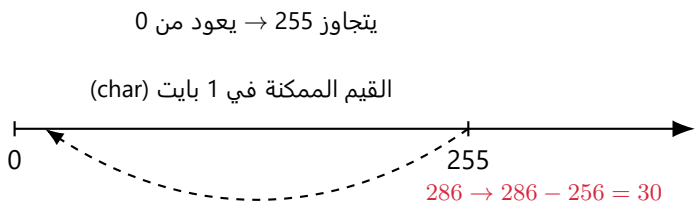
نوع **char** من **8** **بتات** ولا يمكنه أن يحتوي إلا قيمًا من 0 إلى **255**. عندما يتجاوز حساب ما 255، **يعود** **العد** **من** **الصفر**: وتسمى هذه الظاهرة **overflow** (الفيضان).

مثال

$286 - 256 = 30$: القيمة "تدور" بعد الحد الأقصى لـ256 (char قيمة ممكنة، من 0 إلى 255) و**تعود** **من** **الصفر**، لتصل إلى 30. لهذا كانت النتيجة المطبوعة 30 بدلًا من 286.



شكل 1: مخطط انسيابي للكود مع `char totale`: تسلسل خطي من التعليمات، بلا اختيار – لكن النتيجة النهائية خاطئة بسبب overflow.



شكل 2: فيضان نوع char: القيمة التي تتجاوز 255 لا تسبب خطأ، بل "تدور" وتعود من الصفر.

تعريف

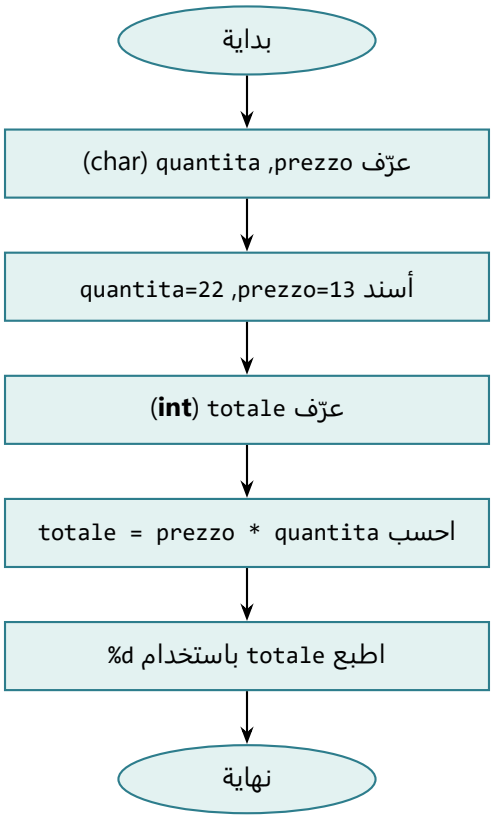
التصحيح: يجب أن يكون **المتغير** الذي يتلقى النتيجة من نوع **يتسع** لها. يكفي هنا تعريف `totale` على أنه `int`:

```
char prezzo, quantita;
prezzo = 13;
quantita = 22;

int totale;
totale = prezzo * quantita;

printf("%d", totale);
```

يمكن أيضًا تعريف `prezzo` و`quantita` على أنهما **`int`**: ليس ضروريًا للحساب، لكنه يجعل الكود أكثر تجانسًا.



شكل 3: مخطط انسيابي للنسخة الصحيحة: التسلسل نفسه، لكن `totale` أصبح `int` ولم يعد الحساب يفيض.

ملاحظة

من يحسب `prezzo * quantita` مباشرة داخل **`printf`**، دون المرور **بالمتغير** `totale`، يحصل أيضًا على النتيجة الصحيحة – لكنها ليست ممارسة جيدة: فبهذه الطريقة **يُعاد تنفيذ الحساب في كل مرة** نطبعه فيها، مما يستهلك قدرة الحساب بلا فائدة. الأفضل الحساب مرة واحدة في **متغير** ثم طباعته.

تنبيه

الخلاصة: انتبه دائمًا إلى **نوع البيانات** وحجم **المتغيرات** التي تستخدمها. من السهل التعود على كتابة **`char`** دائمًا أو **`int`** دائمًا بشكل آلي، ونسيان التحقق مما إذا كانت نتيجة الحساب قد تتجاوز سعة النوع المختار.

3 الأعداد بالفاصلة العشرية: المحدد %f

مثال

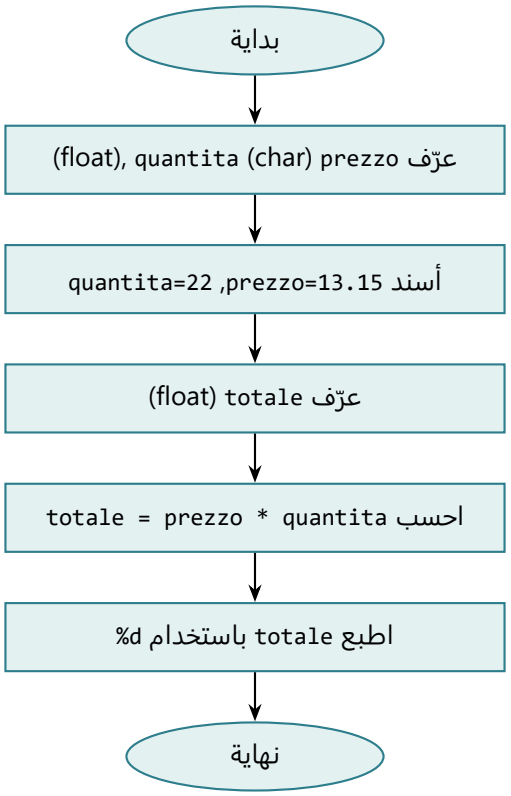
شكل آخر من التمرين: لو كان السعر 13,15 (**بفاصلة عشرية**)، فماذا يتغير؟
الملاحظة الأولى: **المتغير** quantita المعرَّف كـint يشغل 4 **بايت** مع أن نوعًا أصغر يكفي لقيمة مثل 22 – وهذا هدر **للذاكرة** عندما لا نحتاج إلى أعداد كبيرة. النوع المناسب يعتمد دائمًا على **السياق**: الكميات الصحيحة (الأشخاص، القطع) تبقى **أعدادًا صحيحة**؛ والمقادير التي تحتاج إلى كسور (مثل 1 كيلو و22 غرامًا من الدقيق) تتطلب **float**.
التصحيح الصحيح هو وضع **float** على prezzo فقط:

```
float prezzo;
char quantita;

prezzo = 13.15;
quantita = 22;

float totale;
totale = prezzo * quantita;

printf("%d", totale);
```



شكل 4: مخطط انسيابي للشكل الذي فيه prezzo بفاصلة عشرية: التسلسل مطابق، لكن %d عند طباعة float يعطي نتيجة بلا معنى.

تنبيه

مع %d لا معنى للنتيجة المطبوعة، لأن %d يقول لـprintf "هنا يصل عدد **صحيح**": فإذا أعطيناه قيمة **بفاصلة عشرية**، فإن ما يظهر على الشاشة عدد لا علاقة له بالحساب الحقيقي (في مثال القسم: طُبع 289 بدلًا من 289,3، لأن الجزء العشري ضاع في التفسير).

تعريف

لطباعة عدد **بفاصلة عشرية** بشكل صحيح نستخدم **المحدد** %f بدلًا من %d:

```
printf("%f", totale);
```

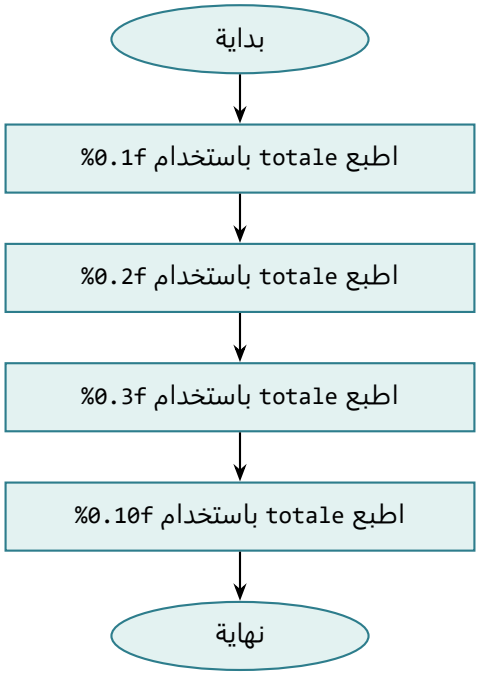
3.1 التحكم في عدد الأرقام بعد الفاصلة

مثال

مع %f وحده تُطبع أرقام كثيرة بعد الفاصلة (يعرض التنسيق الافتراضي ستة). ولتحديد عدد الأرقام المعروضة نكتب بين % وf **السلسلة** .0 متبوعة بعدد الأرقام المطلوب:

```
printf("%0.1f", totale);   // رقم عشري واحد
printf("%0.2f", totale);   // رقمان عشريان
printf("%0.3f", totale);   // ثلاثة أرقام عشرية
printf("%0.10f", totale);  // عشرة أرقام عشرية
```

تعمل هذه الصيغة مع %f فقط: ولا معنى لها مع %d، لأن العدد الصحيح ليس له **أرقام عشرية** لعرضها.



شكل 5: مخطط انسيابي: أربع طباعات متتالية للقيمة نفسها، كل منها بعدد مختلف من الأرقام العشرية.

ملاحظة

القيمة المطبوعة بـ%f **تُقرَّب** إلى الرقم العشري **المختار**، ولا تُقتطع. وهذا سلوك مختلف عن التحويل إلى عدد **صحيح**، الذي يقطع الجزء العشري دون تقريب.

مثال

عدد **الأرقام العشرية** المستخدمة يعتمد على السياق: لسعر منتج يكفي عادةً رقمان (السنتات). أما قياسات المكونات الإلكترونية، التي قد تتطلب قيمًا بـ6 أو 7 أو 8 أرقام بعد الفاصلة، فتحتاج إلى تنسيق بأرقام أكثر (مثل `%0.8f` أو أكثر).

ملاحظة

حادثة في القسم: عند تجربة المثال $22 \times 13{,}15$ مباشرة (النتيجة المتوقعة: $289{,}3$) طبع البرنامج قيمة غريبة بأرقام كثيرة، بدت مرتبطة بأن `quantita` كان لا يزال معرَّفًا كـ`char`. جرب الأستاذ تغيير `quantita` إلى `int` ثم إلى `float`، دون حل المشكلة، واعترف بأنه لا يعرف السبب الدقيق في تلك اللحظة، مفترضًا سلوكًا خاصًا للآلة الافتراضية عبر الإنترنت المستخدمة في الترجمة (ربما مشكلة في تمثيل الفاصلة العائمة، لم تتضح في حينها). ولم تمنع الحادثة من مواصلة شرح التنسيق `%f`.

4 رموز الهروب

تعريف

رمز الهروب (**carattere di escape**) هو سلسلة خاصة تبدأ دائمًا بالرمز **backslash** (\)، لا تُطبع كنص حرفي بل تحدث أثرًا في تنسيق **المخرجات** (**output**).

4.1 \n — سطر جديد

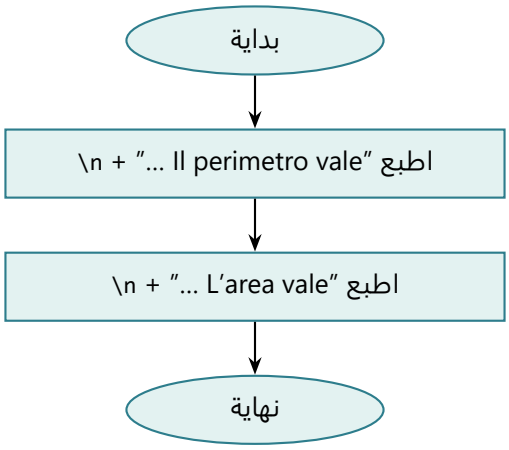
مثال

إذا ظهر \n داخل **سلسلة نصية** (**stringa**) بين علامتي اقتباس، فإن المخرجات **تنتقل إلى سطر جديد** في ذلك الموضع:

```
printf("Il perimetro vale %0.2f\n", perimetro);
printf("L'area vale %0.2f\n", area);
```

تنبيه

كتابة كود C على أسطر مختلفة (مثل `printf` منفصلين، واحد في كل سطر) **لا** تعني تلقائيًا أن المخرجات تنتقل إلى سطر جديد. الانتقال إلى سطر جديد في **الكود المصدري** لا أثر له في **المخرجات**: إذا لم يوجد `\n` داخل السلسلة، تبقى المخرجات على السطر نفسه، حتى لو كان الكود الذي ينتجها على أسطر مختلفة.



شكل 6: مخطط انسيابي: طباعتان متتاليتان، تتبع كلًّا منهما \n التي تنقل المخرجات إلى سطر جديد.

مثال

عرض في القسم: بكتابة "oggi é una" ثم، في **printf** ثانٍ، "bella giornata"، بدون **\n** ولا فراغ بين السلسلتين، تكون **المخرجات** oggi é unabella giornata – كلها ملتصقة، بغض النظر عن تقسيم الكود إلى أسطر. وبإضافة فراغ أو **\n** في نهاية **السلسلة** الأولى تعود النتيجة مقروءة.
يمكن إدخال عدة **\n** متتالية: اثنان **\n** متتاليان ينتقلان إلى سطر جديد مرتين، أي يتركان سطرًا فارغًا بينهما.

4.2 \t — الجدولة

تعريف

\t ينفذ **جدولة** (**tabulazione**): يُدخل عددًا من الفراغات (يمكن ضبطه عادةً في المحرر، وغالبًا 3 أو 4 بحسب المعيار المستخدم).

ملاحظة

لماذا نستخدم **\t** بدلًا من الضغط عدة مرات على مفتاح المسافة؟ لأن **\t** يشغل **بايتًا** **واحدًا** في **الذاكرة**، مقابل 4 بايت لأربعة فراغات مكتوبة واحدًا واحدًا. في نص من بضعة أسطر يكون الفرق ضئيلًا، أما في النصوص الطويلة جدًا (بفراغات متكررة كثيرة) فإن استخدام **الجدولة** قد يقلل كثيرًا مساحة **الذاكرة** المستخدمة.

4.3 \a — إشارة صوتية

تعريف

\a يصدر **إشارة** **صوتية** (beep) عند مصادفته في المخرجات.

مثال

استخدام حقيقي لهذا الرمز: في قارئات الباركود في المتاجر، يشير صوت beep عند المسح إلى نجاح القراءة – وإذا لم يصدر القارئ أي صوت، يفهم العامل فورًا أن شيئًا لم ينجح، دون الحاجة إلى النظر إلى **الشاشة**.

5 تمرين موجَّه: محيط الدائرة ومساحتها

مثال

المسألة (السبورة): بمعرفة نصف القطر r، احسب **محيط** (**perimetro**) الدائرة ومساحتها (**area**) واطبع النتائج.

$$P = 2 \cdot \pi \cdot r$$
$$A = r \cdot r \cdot \pi$$

حيث $\pi = 3{,}141592$.

مثال

الكود المكتوب معًا في القسم:

```
float r;
float PI;

PI = 3.141592;
r = 20;

float perimetro;
float area;

perimetro = 2 * PI * r;
area = r * r * PI;

printf("%0.2f", perimetro);
printf("%0.2f", area);
```

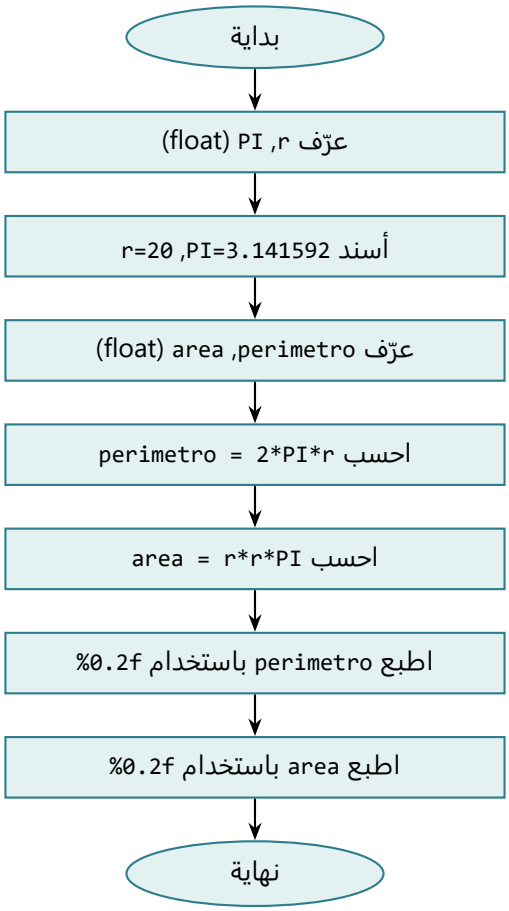
تنبيه

مشكلة القراءة: عند تنفيذ الكود تظهر **المخرجات** **كسلسلة** مشوشة من الأرقام والنقاط (النتيجتان ملتصقتان، مثل 125.661256.64)، لأنه لا يوجد بين **printf** الأول والثاني أي فراغ ولا **\n**. وبإضافة فراغ تصبح النتيجة مقروءة (125.66 1256.64)، لكنها تبقى بلا معنى: فالقارئ لا يعرف أي عدد هو **المحيط** وأيهما المساحة.

تعريف

الحل: إضافة نص وصفي قبل كل **%f**، **لوضع** العدد المطبوع **في** **سياقه**:

```
printf("Il perimetro vale %0.2f\n", perimetro);
printf("L'area vale %0.2f\n", area);
```



شكل 7: مخطط انسيابي لحساب محيط الدائرة ومساحتها: التعريفات والإسنادات والحسابان والطباعتان النهائيتان، كلها بالتسلسل.

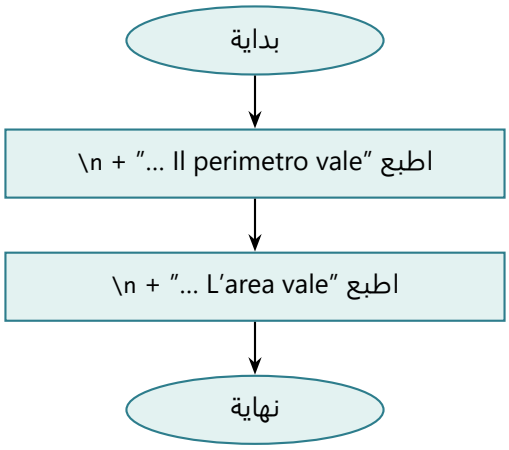
ملاحظة

لماذا من المهم **وضع** الأعداد **في** **سياقها** في **المخرجات**؟ العدد بلا تسمية لا يقول شيئًا لمن يقرؤه، خصوصًا إذا لم يكن القارئ هو من كتب البرنامج. إذا طبع برنامج 125,66 فقط، فلن يعرف سوى كاتبه أنه **محيط**: أما أي شخص آخر يستخدمه (مثل شخص يستخدم البرنامج دون معرفة كوده) فلا سبيل له إلى فهم ذلك. لذلك، في البرامج المعدّة ليستخدمها الآخرون، يجب أن يرافق كل عدد مطبوع نص يشرح معناه.

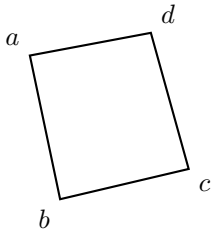
6 تمرين: محيط مضلع من 4 أضلاع

مثال

المسألة (السبورة): لدينا مضلع من 4 أضلاع a, b, c, d (قيمها مسندة مسبقًا)، اكتب برنامجًا يحسب **المحيط** (**perimetro**، مجموع الأضلاع) ويطبعه.



شكل 8: مخطط انسيابي للحل مع السياق: طباعتان متتاليتان، لكل منهما نص يشرح العدد.



شكل 9: الشكل الرباعي المرسوم على السبورة، وأضلاعه الأربعة موسومة بـa وb وc وd.

مثال

الحل، بتطبيق ما رأيناه عن وضع **المخرجات** في سياقها:

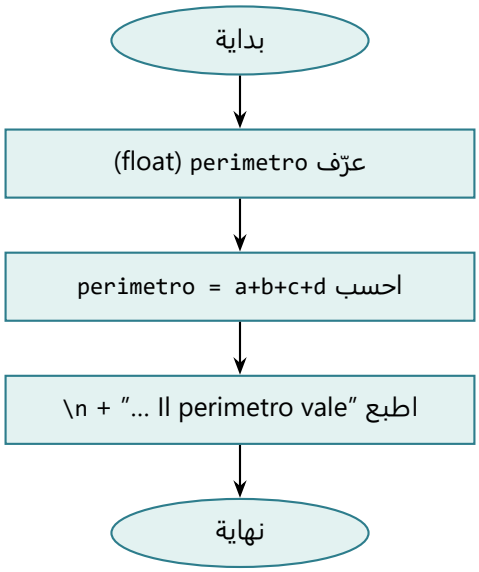
```
float perimetro;
perimetro = a + b + c + d;
printf("Il perimetro vale %0.2f\n", perimetro);
```

7 التمرين الختامي: printf واحد فقط

مثال

المسألة: اكتب برنامجًا يطبع **مستطيلًا** **ممتلئًا** من النجوم، 3 أسطر من 5 رموز، باستخدام `printf` **واحد** **فقط** (لا يُسمح بأكثر من واحد):

```
*****
*****
*****
```



شكل 10: مخطط انسيابي لحساب محيط الشكل الرباعي: التعريف، ومجموع الأضلاع الأربعة، والطباعة مع السياق.

تنبيه

شرط التمرين: يجب أن تنتج كل **المخرجات** من تعليمة **printf** واحدة. وبما أنه لا توجد أعداد للطباعة، فإن الحل هو **سلسلة** نصية واحدة تجمع بين النجوم و**رمز الهروب** **\n** للانتقال من سطر إلى آخر:

```
printf("*****\n*****\n*****\n");
```

مثال

شكل آخر: اطبع المستطيل نفسه لكن **فارغًا** (الحافة فقط من النجوم، والوسط فارغ)، دائمًا بـ**printf** واحد:

```
*****
*   *
*****
```

الحل: الفراغات في السطر الأوسط رموز عادية مثل النجوم، وتُكتب صراحة داخل **السلسلة**:

```
printf("*****\n*   *\n*****\n");
```

ملاحظة

إنه تمرين يلخص كل ما رأيناه في هذا الدرس: رمز الهروب **\n** للانتقال إلى سطر جديد، وبناء **مخرجات** معقدة (متعددة الأسطر) بتعليمة **printf** واحدة، بتركيب **سلسلة** التنسيق حرفًا حرفًا.

8 ما يجب تذكره من هذا الدرس

- **Overflow**: النوع الصغير جدًا (مثل **char**، بحد أقصى 255) لا يشير إلى **خطأ** بل "يدور" و**يعود** **من** **الصفر**، معطيًا نتيجة خاطئة دون تنبيه.

- اختر دائمًا **نوع البيانات** بحسب أقصى قيمة يجب أن يحتويها **المتغير**.
- `%d` للأعداد **الصحيحة**، و`%f` للأعداد **بفاصلة عشرية** – ولا يحل أحدهما محل الآخر.
- `%0.Nf` يتحكم في عدد `N` من **الأرقام العشرية** المطبوعة، مع تقريب القيمة.
- **رموز الهروب** تبدأ بـ`\`: `\n` **ينتقل إلى سطر جديد**، و`\t` يجدول، و`\a` يصدر beep.
- الانتقال إلى سطر جديد **في الكود المصدري** لا يعني الانتقال إلى سطر جديد **في المخرجات**: نحتاج دائمًا إلى `\n` صريح.
- يجب دائمًا **وضع** الأعداد المطبوعة **في سياقها** بنص يشرح معناها.

مثال

للتحضير للدرس القادم: أكمل تمرين **المستطيل** الفارغ من النجوم بـ`printf` واحد، وتحقق أيضًا من حساب **محيط** المضلع ذي الأضلاع الأربعة بمثال عددي **تختاره**.

9 مسرد: كلمات إيطالية تُستخدم في القسم

- **prezzo / quantità / totale**: السعر / الكمية / المجموع
- **a capo**: إلى سطر جديد
- **tabulazione**: جدولة
- **stringa**: سلسلة نصية
- **raggio**: نصف القطر
- **perimetro / area**: المحيط / المساحة
- **rettangolo pieno / vuoto**: مستطيل ممتلئ / فارغ
- **cifre decimali**: الأرقام العشرية
- **Il perimetro vale**: قيمة المحيط هي (نص يطبعه الكود)
- **L'area vale**: قيمة المساحة هي (نص يطبعه الكود)